Bilkent University

Department Of Computer Engineering

Bilkent Information Office Web Page

CS-319 Section 1

Group 2

Deliverable-4 1st Iteration

11.12.2024

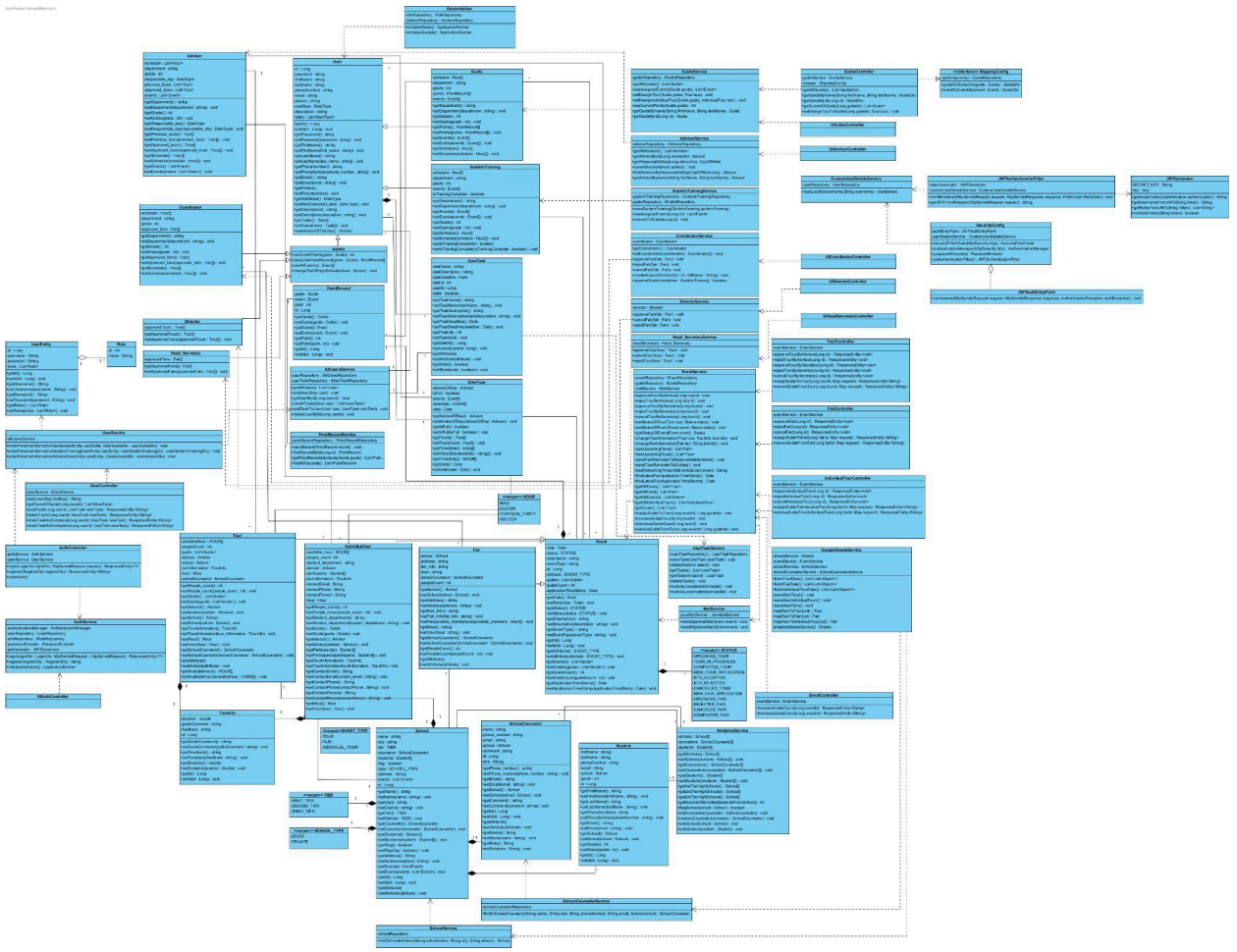
Instructor: Eray Tüzün

Teaching Assistant: Yahya Elnouby

Group Members:

Name	**ID**
Ayça Candan Ataç	22203501
Elif Ceren Çelik	22202534
Elif Ece Can	22201800
Emre Furkan Akyol	22103352
Mustafa Özkan İr	22103267
Ahmet Yağız Sarıdoğan	22202072

1. Detailed Class Diagram



Note: User and UserEntity are separate models because we planned to make the coordinator handle registrations and there are some necessary/not nullable information we want the users to have such as their schedule/email/department etc. The coordinator will create the account as a UserEntity object with just id, password and role and the authentication/authorization is done by UserEntity objects. When the user logs in and fills in the necessary personal info, the User is created and stored in a different table in the database. In conclusion, login and register functions are done using UserEntity models and other functions/attributes are linked to User models.

2. Design Patterns Explained

2.1 Singleton Design Pattern

During the development of our application, the Singleton Design pattern is inherently applied to service and controller classes to ensure that a single instance of each service and controller class is created and made globally accessible throughout the application. This pattern is inherently applied to the project using Spring's @Service, @Controller, and @Autowired annotations.

To elaborate, this design pattern enables efficient resource utilization, as multiple requests can reuse the same instance of a service or controller without repeated instantiations. For example, EventService is a core component responsible for managing event-related operations and interacts with repositories to fetch or update entities. Being a singleton ensures a consistent state and behavior and reuse for all methods. MailService also handles sending emails to school counselors and benefits from singleton behavior by maintaining a unified configuration while serving multiple requests.

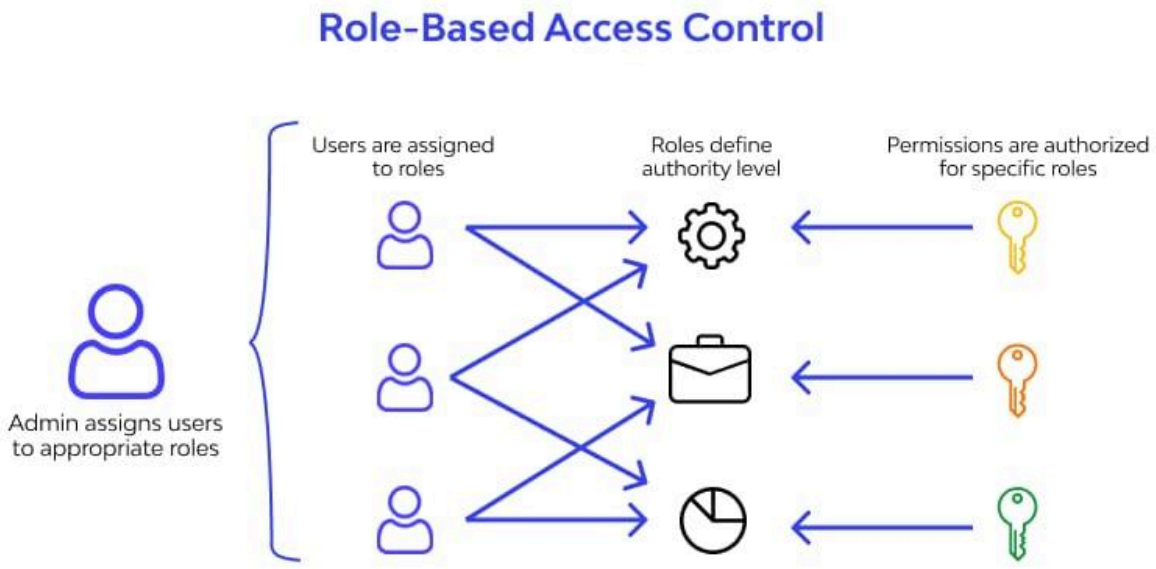
2.2 Adapter Design Pattern

In our software, the GoogleSheetsService class employs the Adapter Design Pattern to bridge the gap between the Google Sheets API and the application's internal data model. The purpose of the GoogleSheetsService is to adapt raw data retrieved from Google Sheets into a format that the application can process. That is, it converts spreadsheet rows into domain objects like Tour, Fair, Individual Tour, School, SchoolCounselor, and Student.

The Adapter Design Pattern can be observed in how GoogleSheetsService acts as an intermediary. It fetches data using the Google Sheets API and maps it to structured objects used by other services like EventService. For example, GoogleSheetsService retrieves tour details from a Google Sheet via fetchTourData(), then transforms them into instances of the Tour entity using the method mapRowToTour(), making them compatible with the rest of the application. Furthermore, during the mapping process, the necessary instances of secondary classes, such as SchoolCounselor and School, are also created using the service methods provided.

This pattern allows the application to remain decoupled from the Google Sheets API. If the API changes, only GoogleSheetsService needs to be updated. By encapsulating the mapping logic within GoogleSheetsService, the Adapter Design Pattern ensures the separation of concerns and enables the integration of external data sources into the application.

2.3 Role Based Access Control



In our application, we endeavored to internalize the Role Based Access Control principles. Although it is not a design pattern in the traditional sense but a security design principle, it's worth mentioning as it provides a structured approach to managing user permissions in the application.

In our implementation, roles such as Advisor, Guide, Coordinator, Director, and Head Secretary are defined with specific permissions according to their responsibilities. For instance, an Advisor can approve or reject tours, while a Head Secretary can finalize approvals or cancel tours. These roles are defined during the initialization stage of the application with the initializeRoles() in the DataInitializer class, and each user is created and associated with the correct role by the application. Therefore, each user's authority level is determined by their associated role. Additionally, the CustomUserDetailsService leverages the UserRepository to load user-specific details, such as roles and permissions, during authentication. This service works with the JWTGenerator to issue tokens, enabling the system to validate permissions.

In short, our project embraced Role-Based Access Control to encapsulate the functionality assigned to each user based on their role. By doing so, we ensured a structured and secure approach to define and enforce user responsibilities.